Efficient Minimum k-Truss Search: A Decomposition-Based Approach

Qifan Zhang, Yang Liu, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Xun Zhou

***Abstract*—Cohesive subgraph mining has been extensively studied and finds numerous graph mining applications such as link farm identification, community detection, and product recommendation. Among various cohesive subgraph structures, the k-truss is particularly notable for its strong structural cohesiveness based on triangles. However, the classical k-truss problem aims to find the k-truss with the maximum number of vertices, which is often extremely large and complex in practice. To fully leverage the benefits of the k-truss, we consider a novel problem called the *minimum k-truss problem*, which seeks to identify a k-truss with the minimum number of vertices, where $k \geq 2$ is a positive integer. We first formally prove the NP-hardness of the problem. We then design a baseline algorithm `MTEnum` that is based on the vertex enumeration and a heuristic method for computing an upper bound. Despite these efforts, `MTEnum` still faces practical efficiency issues which may be due to the fact that the k-truss lacks the hereditary property. To address this issue, we develop a novel decomposition-based framework `DSA`, which elegantly transforms the problem into a sequence of problems that are based on a new cohesive subgraph model called *edge-based s-plex (s-eplex)*. With the hereditary property of s-eplex, we design a branch-and-bound algorithm with several customized techniques for the newly formulated problem. Extensive experiments demonstrate the effectiveness of our studied problem and the efficiency of our proposed algorithm `DSA`. In particular, `DSA` runs up to five orders of magnitude faster than the baseline `MTEnum`.**

***Index Terms*—Cohesive subgraph search, k-truss, branch-and-bound algorithms.**

Received 21 October 2025; revised 21 March 2026; accepted 2 June 2026. Date of publication 8 June 2026; date of current version 8 August 2026. The work of Shengxin Liu was supported in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2025A1515060015, in part by the Shenzhen Science and Technology Program under Grant GXWD20231129111306002. The work of Kaiqiang Yu was supported in part by the "111 Center" under Grant B26023, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM118. The work of Xun Zhou was supported in part by the National Natural Science Foundation of China under Grant 62472125, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2025A1515011258, in part by the Key Technologies R&D Program of Guangdong Province under Grant 2026B0909060001, in part by the Shenzhen Science and Technology Programs under Grant GXWD20231128102922001 and Grant ZDCY20250901111705007. The work of Cheng Long was supported in part by the Ministry of Education, Singapore, under its Academic Research Fund Tier 2 Award under Grant MOE-T2EP20224-0011 and in part by the Tier 1 Award under Grant RG20/24. Recommended for acceptance by X. Yi. *(Corresponding authors: Shengxin Liu; Kaiqiang Yu.)*

Qifan Zhang, Yang Liu, Shengxin Liu, and Xun Zhou are with the Harbin Institute of Technology, Shenzhen 518055, China (e-mail: 21s151127@stu.hit.edu.cn; 23b951003@stu.hit.edu.cn; sxliu@hit.edu.cn; zhouxun2023@hit.edu.cn).

Kaiqiang Yu is with the State Key Laboratory of Novel Software Technology, Nanjing University, Nanjing 210093, China (e-mail: kaiqiang.yu@nju.edu.cn).

Cheng Long is with Nanyang Technological University, Singapore 639798 (e-mail: c.long@ntu.edu.sg).

This article has supplementary downloadable material available at https://doi.org/10.1109/TKDE.2026.3701433, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3701433

I. Introduction

Graphs effectively represent relationships between entities across various domains, such as the World Wide Web, social networks, and biological networks [58]. Cohesive subgraph mining constitutes a fundamental challenge in graph analytics, aiming to detect densely interconnected substructures within real-world networks [15], [31], [33], [71]. The clique model [7], [42], despite being a classical approach to cohesive subgraph mining, imposes rigid constraints, which restricts many practical applications, such as social network analysis, where real-world cohesive structures are often less strict than cliques. Thus, relaxations of the clique concept have been studied to capture the cohesive subgraphs in real applications, such as the k-core [49], k-plex [19], [20], [50], quasi-clique [1], [59], [63], and k-truss [13]. Among these cohesive structures, of particular interest is the k-truss structure due to its strong structural cohesiveness and high computational efficiency [10], [13], [23], [54], [61]. Specifically, the k-truss elegantly softens the clique requirement and requires that every edge belongs to at least $k-2$ triangles in the subgraph, where a triangle contains three vertices that are mutually connected by edges.

Although possessing numerous advantages, the k-truss exhibits certain limitations. One of the major issues might be that the k-truss is defined on the largest subgraph that satisfies the cohesiveness requirement, also known as the *maximum k -truss problem* [13], which is often extremely large and complex in practice. For instance, on the DBLP dataset with 803 K vertices and 3.2 M edges, the maximum 10-truss contains over 85 K vertices, complicating cohesive structure analysis and practical applications. For example, an organizer planning a workshop might use a maximum k-truss search to identify a closely collaborating group of researchers. However, if the k-truss is too large, selecting suitable participants becomes challenging. To mitigate, one might consider using the maximum k-truss for identifying cohesive subgraphs by setting the value of k relatively high, thereby obtaining a smaller k-truss. However, our experiments reveal that, in certain real-world datasets, the maximum k-truss (even with the largest possible value of k) can still be excessively large and lack cohesiveness.

To fully leverage the benefits of the k-truss model, we propose a novel problem called the *minimum k-truss problem* (`MIN`k`TRUSS`). Unlike the classical maximum k-truss problem, which seeks the largest subgraph satisfying the k-truss property,

1041-4347 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

our problem aims to *minimize* the number of vertices while ensuring that the subgraph remains a k-truss—that is, to find the smallest possible k-truss. First, our extensive experiments demonstrate that the minimum k-truss consistently achieves significantly higher density compared to the maximum k-truss. For example, across multiple real-world datasets (see Table III and Fig. 11 in our paper), the density of the minimum k-truss is often greater than 0.9, indicating that almost every possible edge exists between vertices in the subgraph, whereas the maximum k-truss tends to be much larger and less cohesive. This experimental evidence confirms that the minimum k-truss captures much tighter and more strongly connected structures in complex networks. Second, the minimum k-truss is more practical due to its smaller size and high cohesiveness, making it suitable for real-world scenarios such as event organization [47], [48], friend recommendation [6], [44], and protein function identification [3], [4], [14], [22]. For instance, in biological networks, important functional properties—such as similar biological roles among proteins within the same k-truss—are often preserved only in small, highly cohesive subgraphs. Given a protein network and a query protein, the minimum k-truss model can be used to identify other proteins with the same function as the query protein [3], [4], [14], [22], highlighting the utility of the minimum k-truss model in biological research for uncovering meaningful functional modules.

Moreover, Zhang et al. [65] studied the size-constrained k-truss problem (STCS), which requires lower and upper bounds on the size of the community, and returns the k-truss with the largest k within this size constraint. We note that MINkTRUSS offers several advantages over STCS. First, the input parameters for MINkTRUSS are fewer than those required for STCS, providing greater convenience for users. Second, we empirically demonstrate that the minimum k-truss, in most cases, exhibits higher cohesiveness compared to the size-bounded k-truss (see Fig. 11 in Experiments).

Our Baseline Solution: The problem of finding the minimum k-truss is challenging, as we prove its NP-hardness in Section II. Consequently, a viable option for designing practically efficient algorithms is the *branch-and-bound* algorithms. As the k-truss is an edge-induced subgraph, a straightforward branch-and-bound algorithm utilizes the enumeration of edges. That is, we consider all k-trusses by enumerating the *edge sets* and then extract a k-truss with the minimum number of vertices. As the number of edges in real-world graphs is relatively large, we instead design a branch-and-bound algorithm, namely VerEnum, that is based on the *vertex enumeration* strategy. The main idea behind VerEnum is to enumerate vertex sets, (hypothetically) remove disqualifying edges from the current graph, and verify whether the resulting graph qualifies as a k-truss during the enumeration. To improve the performance of our baseline solution, we also provide a polynomial-time heuristic method MTHeur that produces a k-truss for an upper bound of the minimum k-truss. By using the upper bound, we can prune unpromising branches in the branch-and-bound algorithm. We remark that the performance of our proposed baseline MTEnum is still not satisfactory in our experimental studies despite these efforts. One important reason is due to that the k-truss model lacks the hereditary property, which significantly limits its ability to reduce the search space in the branch-and-bound algorithm. We propose VerEnum, a branch-and-bound algorithm based on *vertex enumeration* (Section III). VerEnum enumerates vertex sets, removes disqualifying edges, and verifies whether the resulting graph is a k-truss. To improve performance, we introduce a fast heuristic, MTHeur, which computes a k-truss as an upper bound for the minimum k-truss, enabling pruning of unpromising branches.

Our Decomposition-based Solution: Despite these efforts, the performance of our baseline MTEnum remains unsatisfactory in our experiments. One important reason could be that the k-truss model lacks the hereditary property, which significantly limits its ability to reduce the search space in the branch-and-bound algorithm. To address the practical inefficiency of our baseline method, we develop, in Section IV, a new *decomposition-based search algorithm* (DSA), which elegantly transforms the original MINkTRUSS problem into a sequence of more manageable problems, referred to as the *size-constrained s-eplex problem* (CONSsEPLEX). Specifically, CONSsEPLEX is based on a novel cohesive subgraph model named the *edge-based s-plex (s-eplex)*, where each edge in an s-eplex is allowed to *demolish* at most s triangles, i.e., each edge is not contained in at most s triangles. We note that the s-eplex possesses the desirable *hereditary property*, which facilitates our algorithmic design. As a result, compared to MINkTRUSS, the transformed CONSsEPLEX problems can be efficiently solved by a carefully devised branch-and-bound algorithm, named FastEPX. To further boost the practical performance of FastEPX, we introduce a divide-and-conquer technique, which divides the input graph into several smaller graphs and then applies FastEPX to each one. Finally, we elaborate on how our proposed algorithms can be extended to address personalized search and top-c search variants of the MINkTRUSS problem, enabling broader applications in real-world scenarios.

In addition, we conduct empirical studies to evaluate the effectiveness of the studied problem and the efficiency of our proposed algorithms in Section V. Further, Section VI reviews related work and Section VII concludes the paper.

II. Preliminaries

Let $G = (V, E)$ be a simple graph with $|V| = n$ vertices and $|E| = m$ edges. The subgraph of G induced by the vertex set $S \subseteq V$ is denoted by $G[S]$. We denote the neighbors of vertex v in G as $N_G(v) = \{u|(u, v) \in E\}$ and its degree in G as $d_G(v) = |N_G(v)|$. We use $V(g)$ and $E(g)$ to denote the sets of vertices and edges in a subgraph g, respectively.

A triangle $\triangle(u, v, w)$ is a cycle of length exactly 3 such that $\{(u, v), (v, w), (w, u)\} \in E$. Given an edge (u, v), the set of vertices that can form a triangle with (u, v) in G is denoted by $\Lambda_G(u, v) = \{w|w \in V \text{ and } \triangle(u, v, w) \in G\}$. The *edge support (or support)* $sup_G(u, v)$ of edge (u, v) is defined as the number of triangles in G that contain (u, v), i.e., $sup_G(u, v) = |\Lambda_G(u, v)|$. Note that $sup_G(u, v) \leq n - 2$. Besides, we omit the subscript G when the context is clear. We summerize the notations frequently used throughout the paper in Table I.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF NOTATIONS

Symbol	Description
$G = (V, E)$	Input graph
n, m	$\|V\|$ and $\|E\|$
g	Subgraph of G
$V(g), E(g)$	Vertex and edge sets of g
$N_G(v), d_G(v)$	Neighbors and degree of v in G
$sup_G(e)$	Support of e (number of triangles containing e)
k	Parameter of k-truss: $sup_g(e) \geq k-2$
s	Parameter of s-eplex: $sup_g(e) \geq \|V(g)\| - s - 2$

We next introduce a triangle-based cohesive structure.

Definition 1 ([13]): Given a graph $G = (V, E)$ and an integer $k \geq 2$, a subgraph g of G is said to be a *k-truss* if $sup_g(u, v) \geq k-2$, $\forall (u, v) \in E(g)$.

Among all k-trusses of G, the one containing the largest number of vertices is called the *maximum k-truss*. Note that a k-truss does not have the hereditary property, i.e., a subgraph of a k-truss is not necessarily a k-truss.

Problem (The Minimum k-Truss Problem (MINkTRUSS)): Given a graph $G = (V, E)$ and an integer $k \geq 2$, the *Minimum k-Truss Problem (MINkTRUSS)* aims to find a subgraph g of G which satisfies the following conditions:

1) k-Truss. $\forall e \in E(g), sup_g(e) \geq k-2$.

2) Minimum Cardinality. g is a *non-trivial* subgraph with minimum number of vertices satisfying Condition (1). That is, g contains at least 2 vertices (non-trivial), and we want to minimize $|V(g)|$ such that g satisfies Condition (1).

We remark that we focus on the case when $k \geq 4$. The reason is as follows. For $k = 2$, this problem can be solved trivially, since every edge is an optimal solution. For $k = 3$, the problem corresponds exactly to finding a triangle in the graph (as the edge support of each edge in the triangle is exactly one), which can also be solved in polynomial time. Besides, a subgraph g satisfying Condition (1) is called a *feasible solution* to MINkTRUSS. When g meets both Conditions (1) and (2), g is regarded as an *optimal solution* to MINkTRUSS, and is also called as the *minimum k-truss* in the rest of this paper.

Hardness: We show the NP-hardness of MINkTRUSS.

Theorem 1: The MINkTRUSS problem is NP-hard.

Proof: We reduce the decision version of the maximum clique problem (CLIQUE) to the decision version of the MINkTRUSS problem (KTRUSS) defined as follows.

CLIQUE: Given a graph $G = (V, E)$ and a size bound c, does there exist a clique with at least c vertices?

KTRUSS: Given a graph $G = (V, E)$, an integer k and a size bound c', does there exist a k-truss with at most c' vertices?

Given an instance I of CLIQUE with G and c, we construct an instance I' of KTRUSS with $G' = G, k = c$, and $c' = c$. We next prove that an instance I of CLIQUE is a YES-instance if and only if the constructed instance I' of KTRUSS is a YES-instance.

Case 1: If I of CLIQUE is a YES-instance, there exists a clique of size exactly c vertices, which also directly implies a k-truss of size exactly c' in I' of KTRUSS. By the construction of I', we conclude that I' is a YES-instance.

Algorithm 1: MTEnum.

Input: A graph $G = (V, E)$ and an integer k
Output: The minimum k-truss H
1: $G' = (V', E') \leftarrow$ the maximum k-truss of G via [55];
2: $ub \leftarrow$ MTHeur(G', k); /* Section III-A */
3: $H \leftarrow$ VerEnum$(G', \emptyset, V(G'), ub)$; /* Section III-B */
4: **return** H

Case 2: If I' of KTRUSS is a YES-instance, there is a k-truss of size at most c' vertices. According to the definition of k-truss, we have: 1) a k-truss has at least k vertices; 2) a k-truss with exactly k vertices is a clique. Based on our construction of I', there is a c-truss of size at most c since we have $k = c' = c$ in I'. Moreover, by the above 1) and 2), we deduce that there is a clique of size c in I which means that I is a YES-instance. Thus the proof is complete. □

III. AN ENUMERATION-BASED BASELINE SOLUTION

In this section, we present a baseline algorithm for the MINkTRUSS problem. The framework of our baseline method is shown in Algorithm 1. We first remove unpromising edges/vertices to obtain a maximum k-truss $G' = (V', E')$ using the method in [55] (Line 1) (note that the minimum k-truss must be contained in the maximum k-truss since otherwise a larger k-truss can be obtained by combining them together). After obtaining the reduced graph, we heuristically compute the size of a feasible solution (MTHeur), which serves as an upper bound (denoted as ub), and this upper bound will be applied in the enumeration stage for pruning (Line 2). Finally, we execute the vertex enumeration method (VerEnum) for searching the optimal solution (Line 3). The details of MTHeur and VerEnum are discussed in Sections III-A and III-B, respectively.

A. A Heuristic Method

We propose a heuristic solution named MTHeur to obtain an upper bound for the minimum k-truss (i.e., a feasible k-truss). The general idea is to continuously add vertices to an initially empty vertex set while concurrently removing undesirable edges from its induced subgraph. The detailed procedure for MTHeur is in Algorithm 2. Specifically, we first initialize necessary variables (Line 1). In this process, we first add to the selected set S the vertices of the edge (u, v), which has the maximum support in G. Simultaneously, we define the candidate set U as the set of remaining vertices in V, excluding u and v. Additionally, we initialize the heuristic solution H as the induced subgraph $G[S]$. The subsequent procedure involves continuously adding vertices to S and dynamically updating H, until H qualifies as a valid k-truss (Lines 2-5). Our approach is to find a vertex w that forms the largest number of triangles in $G[S \cup \{w\}]$ (Line 3). The intuition is that w can potentially be included in a denser subgraph. Following this, we add w to S, assign $G[S]$ to H, and update U (Line 4). Moreover, we need to remove unqualified edges in H since it may not satisfy the requirement of k-truss (Line 5). Particularly, if H is a non-empty k-truss, we break the loop and return the size of H as the upper bound (Line 6).

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: `MTHeur`.

Input: A graph $G = (V, E)$ and an integer k
Output: An upper bound ub of the minimum k-truss
1 Initialize the selected set $S \leftarrow \{u, v\}$ where edge (u, v) has the maximum support in G; $U \leftarrow V \setminus \{u, v\}$; $H \leftarrow G[S]$;
2 **while** H *is a non-empty* k*-truss* **do**
3 Choose the vertex w in U that forms the largest number of triangles in $G[S \cup \{w\}]$;
4 $S \leftarrow S \cup \{w\}$; $H \leftarrow G[S]$; $U \leftarrow U \setminus \{w\}$;
5 Remove edges whose supports are less than $k-2$ in H;
6 **return** $|V(H)|$;

Algorithm 3: `VerEnum`.

Input: A graph $G = (V, E)$, the selected set S, the candidate set U, and the upper bound ub
Output: The minimum k-truss H
1 $H \leftarrow \emptyset$; `// maintained globally`
2 `Enum-Rec`(G, S, U, ub);
3 **return** H;

Procedure: `Enum-Rec`(G, S, U, ub)
4 **if** $|S| \geq ub$ **then**
5 **return**;
6 $G_{temp} \leftarrow G[S]$;
7 Remove edges whose supports are less than $k-2$ in G_{temp};
8 **if** G_{temp} *is a non-empty* k*-truss* and $|V(G_{temp})| < ub$ **then**
9 $ub \leftarrow |V(G_{temp})|$; $H \leftarrow G_{temp}$;
10 **return**;
11 **if** $U \neq \emptyset$ **then**
12 $v \leftarrow$ randomly select a vertex from U;
13 `Enum-Rec`$(G, S \cup \{v\}, U \setminus \{v\}, ub)$;
14 `Enum-Rec`$(G, S, U \setminus \{v\}, ub)$;

Time Complexity: The complexity of Algorithm 2 is primarily determined by Lines 3 and 5 of the while-loop. For Line 3, we can utilize a self-balancing binary search tree to store the numbers of triangles in $G[S \cup \{w\}]$ for each $w \in U$. Since there are m edges in total, there are at most m updates in the binary search tree where each update requires $O(n \log n)$ time. Thus, the total cost of Line 3 is $O(nm \log n)$. For one execution of Line 5, the truss decomposition algorithm runs in $O(m^{1.5})$ time [55]. Since the number of iterations of while-loop is at most n, the total cost for Line 5 is thus $O(nm^{1.5})$. Hence, the time complexity of Algorithm 2 is $O(nm^{1.5})$.

B. A Vertex-Based Enumeration Method

We present our vertex-based enumeration algorithm in Algorithm 3 (`VerEnum`) for finding the minimum k-truss. The main idea of `VerEnum` is to enumerate the vertex sets (instead of the straightforward method that enumerates the edge sets), remove unqualified edges in the current graph, and check whether the remaining graph is a qualified k-truss during the enumeration. To begin with, `VerEnum` initializes a global variable H to record the current solution during the enumeration process `Enum-Rec` and returns H after finishing `Enum-Rec` (Lines 1-3). The recursive process of `Enum-Rec` (Lines 4-14) is as follows. If the candidate set U is not empty, a random vertex v is selected and deleted from U, and each execution of `Enum-Rec` generates two recursive calls: (1) S includes v from U (Line 13), and (2) U excludes v in the current graph (Line 14). Then, in each call of `Enum-Rec`, we can prune the current recursive branch if the size of the selected set $|S|$ is larger than ub (Lines 4-5). Then, we utilize a copy G_{temp} of $G[S]$ and iteratively remove unqualified edges from G_{temp} (Lines 6-7). If the current G_{temp} is a non-empty k-truss (i.e., its number of edges is non-zero) and $|V(G_{temp})| < ub$, we record G_{temp} as H and update its size as the upper bound (Lines 8-10).

Time Complexity: Obviously, the number of recursive calls to `Enum-Rec` is at most 2^n, and each execution of `Enum-Rec` costs $O(m^{1.5})$ due to the truss decomposition in Line 7 [55]. Thus, the time complexity of Algorithm 3 is $O(2^n \cdot m^{1.5})$.

Example: Consider an example in Fig. 1. First, we assume the current $ub = 6$ and $S = \emptyset$ in Fig. 1(a), and our goal is to find a minimum 4-truss. To achieve this, we recursively add vertices and their corresponding edges into $G[S]$, store the graph as G_{temp}, remove its unqualified edges, and check whether the remaining graph can form a 4-truss, as shown in Fig. 1(b)–(h). Following this procedure, we add v_5 and edges $\{(v_2, v_5), (v_3, v_5), (v_4, v_5)\}$ into S, as shown in Fig. 1(f). Then we delete edges $\{(v_0, v_1), (v_0, v_2), (v_1, v_3)\}$, since their edge supports are all less than $k - 2 = 2$. Additionally, a 4-truss with size smaller than the previous $ub = 6$ is obtained (shown in Fig. 1(g)), and we update $ub = 4$.

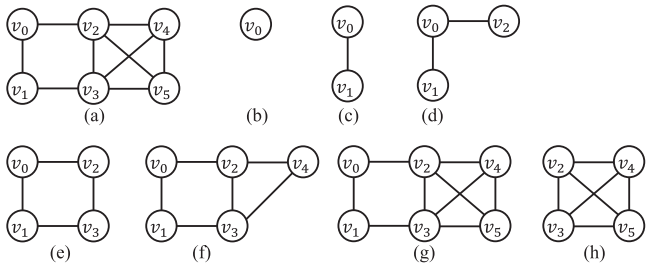


Fig. 1. An example of our vertex-based enumeration method `VerEnum`.

Space Complexity: The space complexity of `VerEnum` is bounded by $O(|V| + |E|)$, which is dominated by that of the recursive process with the depth of $O(|V|)$.

C. Limitations of the Baseline Method `MTEnum`

Despite achieving good performance through a novel heuristic approach and a carefully designed enumeration procedure, the baseline method `MTEnum` still has potential limitations that can affect its practical performance.

Large Candidate Set: Although the candidate set has been reduced using the maximum k-truss in Line 1 of Algorithm 1, a large number of vertices still remain for the vertex enumeration procedure `VerEnum` (Line 3 of Algorithm 1). Given that the (worst-case) time complexity of the baseline `MTEnum` is exponential related to the number of candidate vertices used

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

in the vertex enumeration procedure VerEnum, the size of the candidate vertex set clearly affects the practical efficiency.

Lack of Effective Termination Rules: The vertex-based enumeration method VerEnum examines all combinations of vertex sets, which results in at most 2^n recursive calls to Enum-Rec, where n is the size of input graph to VerEnum. Thus, early termination rules are of great importance to achieve practical efficiency. However, VerEnum lacks of effective early termination rules due to the absence of the hereditary property of k-truss.

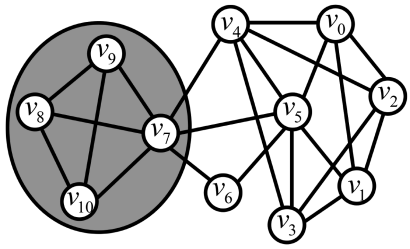


Fig. 2. Example graph.

IV. Our Decomposition-Based Approach

In this section, we design a decomposition-based framework to solve the MINkTRUSS problem. Specifically, in Section IV-A, we introduce the elegant *decomposition-based* framework DSA for the MINkTRUSS problem based on a new edge-based cohesive subgraph structure. We then propose a novel branch-and-bound method for efficiently solving the newly formulated problem in Section IV-B. Finally, Section IV-C presents additional efficiency boosting techniques and adaptations of our algorithms for various variants.

A. MINkTRUSS: A Decomposition-Based Framework

The idea of our decomposition-based framework is to *decompose* the MINkTRUSS problem into a series of newly formulated problems, namely *size-constrained s-eplex problems*, which retain some nice properties and thus can be solved effectively with carefully-designed reduction rules. We first define a new edge-based cohesive subgraph structure.

Definition 2. (edge-based s-plex (s-eplex)): Given a graph G and an integer s, a subgraph g of G is said to be *s-eplex* if and only if (1) g contains at least 1 edge, i.e., $|E(g)| \geq 1$; and (2) each edge is contained in at least $|V(g)| - s - 2$ triangles, i.e., $\forall e \in E(g), sup_g(e) \geq |V(g)| - s - 2$.

By Definition 2, we note that (1) an s-eplex contains at least 1 edge; and (2) each edge in an s-eplex is allowed to *demolish* at most s triangles inside (note that there are at most $|V(g)| - 2$ triangles in a subgraph g). Clearly, 0-eplex reduces to clique. Besides, we observe that s-eplex retains one nice property, namely *hereditary property*, which will facilitate our algorithmic design.

Lemma 1. (Hereditary): If G is an s-eplex, any (vertex) induced subgraph $g \subseteq G$ with $|E(g)| \geq 1$ is also an s-eplex.

Lemma 1 follows directly, as removing any vertex from an s-eplex does not increase the size of vertices that cannot form a triangle with any edge e in the remaining graph.

k-truss vs. s-eplex: We note that s-eplex is defined based on the maximum number of missing triangles (say, $sup_g(e) \geq |V(g)| - s - 2$), while k-truss is characterized by the minimum number of connected triangles (say, $sup_g(e) \geq k - 2$). As a result, the hereditary property holds for s-eplex but does not for k-truss (e.g., a vertex induced subgraph of a k-truss may no longer be a k-truss).

Example of the non-hereditary property of k-truss: Consider an example in Fig. 2. $G[\{v_0, v_1, v_2, v_3, v_4, v_5\}]$ is a 2-eplex, and its induced subgraph $G[\{v_0, v_1, v_2, v_3, v_4\}]$ is also a 2-eplex due to the hereditary property of s-eplex. $G[\{v_0, v_1, v_4, v_5, v_6, v_7\}]$ is not a 2-eplex since the edge support of (v_0, v_1) is less than $6 - 2 - 2 = 2$. On the other hand, $G[\{v_7, v_8, v_9, v_{10}\}]$ is a 4-truss, while its induced subgraph $G[\{v_7, v_8, v_9\}]$ is not a 4-truss since in this subgraph each edge has support only 1, but a 4-truss requires the support of every edge to be at least 2. This illustrates that k-truss does not possess the hereditary property.

Besides, we can easily deduce the following observation.

Lemma 2: A k-truss g_1 is an s'-eplex with $s' = |V(g_1)| - k$. An s-eplex g_2 is a k'-truss with $k' = \max\{0, |V(g_2)| - s\}$.

Proof: Let g be a subgraph with $|V(g)|$ vertices.

1) Suppose g is a k-truss. By definition, for every edge $e \in E(g)$ we have $\text{sup}_g(e) \geq k - 2$. Since the support of any edge is at most $|V(g)| - 2$, if we set $s = |V(g)| - k$, then the s-eplex condition requires $\text{sup}_g(e) \geq |V(g)| - s - 2 = |V(g)| - (|V(g)| - k) - 2 = k - 2$, which is exactly the same. Hence every k-truss is also a $(|V(g)| - k)$-eplex.

2) Suppose g is an s-eplex. By definition, for every edge $e \in E(g)$ we have $\text{sup}_g(e) \geq |V(g)| - s - 2$. Let $k' = |V(g)| - s$. Then the k'-truss condition requires $\text{sup}_g(e) \geq k' - 2 = (|V(g)| - s) - 2$, which coincides with the s-eplex condition. Thus g is a k'-truss. If $|V(g)| - s \leq 0$, we define $k' = \max\{0, |V(g)| - s\}$ to avoid trivial cases.

Therefore a k-truss corresponds to a $(|V(g)| - k)$-eplex and an s-eplex corresponds to a $(|V(g)| - s)$-truss. □

Thus, if one aims to find a k-truss with size equal to a threshold l, it is equivalent to find a $(l - k)$-eplex g with the size equal to l.

Divide MINkTRUSS into CONSsEPLEX: The MINkTRUSS problem can be divided into a sequence of $(|V| - k + 1)$ subproblems; the i-th $(0 \leq i \leq |V| - k)$ subproblem aims to find a k-truss with the size equal to $i + k$ (note that a k-truss has the size at least k and at most $|V|$); and clearly, the minimum k-truss is the smallest one among all found subgraphs. Furthermore, the i-th subproblem can be equivalently transformed into the problem of finding an i-eplex with the size equal to $i + k$ based on Lemma 2. Formally, we formulate the latter problem as below.

Problem. (The Size-Constrained s-Eplex Problem (CONSsEPLEX)): Given a graph G and two integers s and k, the *size-constrained s-eplex problem* finds a *connected* subgraph g of G which satisfies the following conditions:

(1) s-eplex. $\forall e \in E(g), sup_g(e) \geq |V(g)| - s - 2$.

(2) Size Constraint. $|V(g)| = s + k$.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Our Proposed Framework: Decomposition-based Search Algorithm (DSA).

Input: A graph $G = (V, E)$ and an integer k
Output: The minimum k-truss
1 $G' = (V', E') \leftarrow$ the maximum k-truss of G via [55];
2 **if** $G' = \emptyset$ **then return** $\emptyset$;
3 **for** $s = 0, 1, 2, ..., |V'| - k$ **do**
4 $\quad H \leftarrow$ Solve CONSsEPLEX with a specific s;
5 $\quad$ **if** $H \neq \emptyset$ **then return** H;

We first note that the CONSsEPLEX problem requires the returned graph to be connected, e.g., there should be no zero-degree vertices. Therefore, we can decompose the MINkTRUSS problem into a sequence of $(|V| - k + 1)$ CONSsEPLEX problem instances; and the i-th $(0 \leq i \leq |V| - k)$ instance has the parameter $s = i$ by the above discussion. Based on this, we then propose a new decomposition-based framework DSA. Below, we elaborate on the details.

A new framework DSA:Our decomposition-based search algorithm (DSA) is given in Algorithm 4. First, it refines the input graph G to its maximum k-truss (Line 1) via truss decomposition ([55]) based on the same reason mentioned in Algorithm 1. Besides, if G' is empty, we can deduce that there does not exist any k-truss, and thus can terminate (Line 2). Second, it constructs $(|V'| - k + 1)$ CONSsEPLEX problem instances as discussed above (Lines 3-5). We remark that (1) when H is empty for a parameter s in Line 4, it means that no qualified constrained s-eplex is found, or equivalently, the size of minimum k-truss is larger than $s + k$; (2) it can terminate in Line 5 once a non-empty subgraph H is found (since we aim to find the minimum k-truss and all subgraphs found by the following instances become larger); and (3) if G' is non-empty, it can always return a k-truss since G' is a $(|V'| - k)$-eplex and can be returned by the last instance (i.e., $s = |V'| - k$) in the worst case. In summary, the correctness of DSA can be easily verified, and the time complexity is given in the following section.

Benefits of DSA:Compared to the original MINkTRUSS problem, the newly transformed CONSsEPLEX problem can be solved efficiently via exploiting the hereditary property of s-eplex. Specifically, we will design novel branch-and-bound methods for CONSsEPLEX, namely FastEPX and DCFastEPX (Line 4 of Algorithm 4), which incorporates novel branching strategies, effective pruning rules, and tight upper bounds. The details of FastEPX and DCFastEPX are introduced in the following sections.

Remark: We note that the definition of s-eplex is similar to that of s-plex (say, a subgraph where every vertex is not adjacent to at most s vertices including itself [50]). We also observe that the relationship between k-truss and s-eplex is analogous to that between k-core and s-plex. That is, the k-core is defined based on the minimum number of adjacent vertices, while the s-plex is defined based on the maximum allowable number of missing adjacent vertices. Nonetheless, existing methods for finding maximum s-plex ([8], [9], [18], [28], [29], [57], [60], [70]) cannot be directly adapted to solve the CONSsEPLEX problem due to the following lemma.

Lemma 3: An s-eplex is an $(s + 1)$-plex. However, an $(s + 1)$-plex might not be an s-eplex.

Proof: Given an s-eplex g, by the definition of s-eplex, $sup_g(e) \geq |V(g)| - s - 2$, $\forall e \in E(g)$. Then, $d_g(v) \geq |V(g)| - s - 1$, $\forall v \in V(g)$ which implies that g is an $(s + 1)$-plex. The second part can be obtained by a counter-example in Fig. 2. The induced subgraph g' with $\{v_0, v_1, v_3, v_4, v_5\}$ is a 2-plex but not a 1-eplex since edge (v_0, v_1) is only contained in one triangle in g' which is less than $5 - 1 - 2 = 2$. □

B. CONSsEPLEX: A Branch-and-Bound Method

In this part, we introduce an efficient branch-and-bound algorithm, namely FastEPX, for solving the newly formulated CONSsEPLEX problems. Specifically, it recursively partitions the problem instance (of finding a size-constrained s-eplex in G) to multiple sub-instances (of finding a size-constrained s-eplex in a *smaller* subgraph of G) via *branching*. Each sub-instance corresponds to a *branch*, which is represented by a pair (S, U), where

- **Partial solution** S is an s-eplex found currently, which is a subgraph of G induced by a set of edges $E(S) \subseteq E(G)$.
- **Candidate set** U is a set of vertices that will be considered to be included into S to form a larger s-eplex.

Solving an instance or branch (S, U) means finding a size-constrained s-eplex H (in the branch) such that (1) H is a connected subgraph of $G[V(S) \cup U]$, and (2) H contains all vertices in $V(S)$ and $H[V(S)]$ is a subgraph of S (i.e., edges in $H[V(S)]$ is a subset of that in S, formally, $E(H[V(S)]) \subseteq E(S)$). Thus, the branch (S, U) covers all size-constrained s-eplexes, each satisfying the above two constraints; and solving branch $(\emptyset, V)$ finds a size-constrained s-eplex in G, thus solving CONSsEPLEX.

To solve a branch (S, U), we adopt a binary branching strategy. Specifically, it partitions the branch (S, U) as two sub-branches based on a vertex v selected from U: sub-branch $(S \cup v, U \setminus \{v\})$ that includes v into the found s-eplex (in this sub-branch), and sub-branch $(S, U \setminus \{v\})$ that excludes v from the found s-eplex. Here, $S \cup v$ represents the resulting graph by adding v to $V(S)$ and adding all edges between v and $V(S)$ in G to $E(S)$, i.e.,

$$S \cup v = (V(S) \cup \{v\}, E(S) \cup \{(v, u) \in E \mid \forall u \in V(S)\}). \quad (1)$$

Besides, one additional process, namely RefineToEPX, is required for the newly formed sub-branch $(S \cup v, U \setminus \{v\})$ since the partial solution $S \cup v$ could not be an s-eplex (note that simply terminating the sub-branch could lose the solution in this case). Specifically, RefineToEPX$(S \cup v)$ returns a subgraph, denoted by $\mathcal{R}(S \cup v)$, of $S \cup v$ that (1) contains $V(S) \cup \{v\}$ and (2) is an s-eplex, by iteratively removing from $S \cup v$ those edges that violate the definition of s-eplex, i.e., with the edge support less than $|V(S \cup v)| - s - 2$. We then update the branch $(S \cup v, U \setminus \{v\})$ by replacing $S \cup v$ with the subgraph $\mathcal{R}(S \cup v)$, i.e., $(\mathcal{R}(S \cup v), U \setminus \{v\})$. We show the correctness of the above branching strategy (i.e., a size-constrained s-eplex H in (S, U) can still be found from the newly formed sub-branches).

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: `FastEPX`.

Input: A graph $G = (V, E)$ and two integers s and k
Output: A size-constrained s-eplex H if exists; $\emptyset$ otherwise

1 $H \leftarrow$`FastEPX-Rec`$(G, (\emptyset, \emptyset), V)$;
2 **return** H;

Procedure: `FastEPX-Rec`(G, S, U)
`/* Termination */`
3 **if** $|V(S)| = s + k$ **then return** S;
4 **if** $|V(S) \cup U| < s + k$ **then return** $\emptyset$;
5 **if** $S \cup G[U]$ *is a connected* s*-eplex* **then**
6 $H \leftarrow$ a $(k + s)$-connected-subgraph of $S \cup G[U]$;
7 **return** H;
`/* Branching */`
8 Select vertex v with the smallest degree in $G[U]$;
9 $\mathcal{R}(S \cup v) \leftarrow$`RefineToEPX`$(S \cup v)$;
10 **if** $E(\mathcal{R}(S \cup v)) \neq \emptyset$ **then**
11 `FastEPX-Rec`$(G, \mathcal{R}(S \cup v), U \setminus \{v\})$
12 `FastEPX-Rec`$(G, S, U \setminus \{v\})$;

Procedure: `RefineToEPX`(S)
13 **while** S *is not an* s*-eplex* ***and*** $E(S)$ *is not empty* **do**
14 $e^* \leftarrow$ the edge with the smallest support in S;
15 $S \leftarrow (V(S), E(S) \setminus \{e^*\})$;
16 **return** S;

Lemma 4: Given a branch (S, U) and a vertex v in U, solving sub-branches $(\mathcal{R}(S \cup v), U \setminus \{v\})$ and $(S, U \setminus \{v\})$ solves (S, U).

Proof: In general, there are two cases. First, if (S, U) does not cover any size-constrained s-eplex (i.e., solving (S, U) will return an empty graph), we can easily verify that $(\mathcal{R}(S \cup v), U \setminus \{v\})$ and $(S, U \setminus \{v\})$ do not cover any size-constrained s-eplex either since all s-eplexes in the formed two sub-branches are covered by (S, U). To verify this, consider an s-eplex H found in $(\mathcal{R}(S \cup v), U \setminus \{v\})$. We can derive that H is also in (S, U) since (1) H is a connected subgraph of $G[V(S) \cup U]$ (note that H is a connected subgraph of $G[V(\mathcal{R}(S \cup v) \cup U \setminus \{v\})]$ which is equal to $G[V(S) \cup U]$), (2) H contains all vertices in S (note that H contains all vertices in $V(\mathcal{R}(S \cup v))$ which is equal to $V(S) \cup \{v\}$), and (3) $H[V(S)]$ is a subgraph of S (note that $H[V(\mathcal{R}(S \cup v))]$ is a subgraph of $\mathcal{R}(S \cup v)$ and $\mathcal{R}(S \cup v) \setminus v$ is a subgraph of S.) We can similarly verify that an s-eplex found in $(S, U \setminus \{v\})$ is also in (S, U).

Second, if (S, U) holds a size-constrained s-eplex H, we then verify that a size constrained s-eplex H' with $V(H') = V(H)$ will be found in either $(\mathcal{R}(S \cup v), U \setminus \{v\})$ (if H contains vertex v) or $(S, U \setminus \{v\})$ (if H excludes vertex v), and thus solving two formed sub-branches solves (S, U) (note that solving a branch seeks to find one size constrained s-eplex instead of finding all of them). In specific, assume that H contains vertex v.

Let $g = H[V(S) \cup \{v\}]$. Clearly, both $\mathcal{R}(S \cup v)$ (i.e., a subgraph returned by `RefineToEPX`) and g are subgraphs of $S \cup v$. We then derive that $E(\mathcal{R}(S \cup v))$ is a superset of $E(g)$, i.e., $E(g) \subseteq E(\mathcal{R}(S \cup v))$ by contradiction. Suppose that $\mathcal{R}(S \cup v)$ is not a superset of g, i.e., $E(g) \setminus E(\mathcal{R}(S \cup v))$ is not empty. Consider the procedure `RefineToEPX`$(S \cup v)$ where some edges in $S \cup v$ are removed at the certain ordering, i.e., $\{e_1, e_2, \ldots, e_c\}$ such that e_i $(1 \leq i \leq c \leq E(S \cup v))$ is removed at the i-th round. Clearly, $\{e_1, e_2, \ldots, e_c\} \cap E(g)$ is not empty based on our assumption. let e_i $(1 \leq i \leq c)$ be the first edge contained in g, i.e, $e_i \in E(g)$ and $E(g) \cap \{e_1, e_2, \ldots, e_{i-1}\} = \emptyset$. Let g_i be the subgraph of $S \cup v$ obtained by removing from $S \cup v$ all edges in $\{e_1, e_2, \ldots, e_{i-1}\}$. Clearly, e_i has the support in g no larger than that in g_i, i.e.,

$$sup_g(e_i) \leq sup_{g_1}(e_i), \tag{2}$$

since g is a subgraph of g_i. Note that g_i is not an s-eplex (since otherwise g_i will be returned by `RefineToEPX`$(S \cup v)$) and thus e_i has the support in g_i smaller than $|V(S \cup v)| - s - 2$. Hence, e_i has the support in g smaller than $|V(S \cup v)| - s - 2$ and g is also not an s-eplex, which contradicts to the fact that $g = H[V(S) \cup \{v\}]$ as an induced subgraph of an s-eplex H is an s-eplex due to the hereditary property.

Let H' be a graph obtained by adding to H all edges in $E(\mathcal{R}(S \cup v)) \setminus E(g)$, i.e.,

$$H' = (V(H), E(H) \cup E(\mathcal{R}(S \cup v))). \tag{3}$$

Clearly, H' is also an s-eplex since H is an s-eplex and all edges in H' has the support no smaller than that in H. Finally, we can derive that H' is in $(\mathcal{R}(S \cup v), U \setminus \{v\})$ since (1) H' is a connected subgraph of $G[V(S) \cup U]$, (2) H' contains all vertices in $S \cup \{v\}$, and (3) $H'[V(S) \cup \{v\}]$ is a subgraph of $\mathcal{R}(S \cup v)$ (note that $H'[V(S) \cup \{v\}]$ is equal to $\mathcal{R}(S \cup v)$ based on the construction of H' and the fact that $H[V(S) \cup \{v\}]$ is a subgraph of $\mathcal{R}(S \cup v)$). □

Finally, we show our `FastEPX` with the aforementioned branching method in Algorithm 5. Specifically, it has two stages: `Termination` and `Branching`. In `Termination` (Lines 3-7), it terminates a branch in the following cases. First, when $|V(S)| = s + k$, the current S is a solution to `CONS`s`EPLEX` (Line 3). Second, when $|V(S) \cup U| < s + k$, we can prune this branch since the size of the largest s-eplex in the remaining graph cannot reach $s + k$ (Line 4). Third, when $S \cup G[U]$ (where $|V(S) \cup U| \geq s + k$) is a connected s-eplex, we can extract a solution, i.e., a $(k + s)$-connected-subgraph of $S \cup G[U]$, to `CONS`s`EPLEX` by first constructing a spanning tree of $S \cup G[U]$ and iteratively removing the leaves of this spanning tree until the size of the remaining graph reaches $s + k$ (Lines 5-7). The correctness follows Lemma 1. In `Branching` (Lines 8-12), based on Lemma 4, our algorithm selects a vertex v with the minimum degree in $G[U]$, calls `RefineToEPX`$(S \cup v)$ and generates two branches based on v: $(R(S \cup v), U \setminus \{v\})$ and $(S, U \setminus \{v\})$. Note that the former one is generated conditionally when $E(\mathcal{R}(S \cup v)) \neq \emptyset$ since otherwise, the newly formed partial solution $R(S \cup v)$ will lose the property of s-eplex.

Time Complexity: It is clear to see that the number of recursive calls to `FastEPX` is at most 2^n, and each execution of `FastEPX` costs $O(m^{1.5})$ due to `RefineToEPX` performs a truss decomposition using $O(m^{1.5})$ ([55]) in Lines 13-15. Thus, the time complexity of Algorithm 5 is $O(2^n \cdot m^{1.5})$.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

C. CONSsEPLEX: *Efficiency Boosting Techniques*

To boost the practical performance of FastEPX, we then introduce the *divide-and-conquer* technique, which divides the input graph into several smaller ones and runs FastEPX on each of them. We remark that while the divide-and-conquer technique has been widely used for finding cohesive subgraphs [8], [9], [63], [64], it is the first time and no trivial to apply it for finding the minimum k-truss. Specifically, we observe the following properties of our proposed s-eplex, which enable the divide-and-conquer technique.

Lemma 5: Let g be an s-eplex with size at least $2s+1$. Any two non-adjacent vertices in g must have at least $|V(g)|-2s$ common neighbors, and any two adjacent vertices in g must have at least $|V(g)|-2s-2$ common neighbors.

Proof: Given an s-plex g of size $|V(g)| \geq 2s-1$, we know that (1) any two non-adjacent vertices in g must have at least $|V(g)|-2s+2$ common neighbors, and (2) any two adjacent vertices must have at least $|V(g)|-2s$ common neighbors [70]. Since an s-eplex is also an $(s+1)$-plex by Lemma 3, we then have any two non-adjacent vertices in g must have at least $|V(g)|-2s$ common neighbors, and any two adjacent vertices in g must have at least $|V(g)|-2s-2$ common neighbors. □

Lemma 6 (Two-Diameter): The diameter of an s-eplex with size at least $2s+1$ is at most 2.

Proof: By Lemma 5, any two non-adjacent vertices in g must have at least $|V(g)|-2s$ common neighbors. Therefore, in an s-eplex with size at least $2s+1$, any two vertices must have at least one common neighbor, which implies that the distance between these two vertices is at most 2. □

Consider a problem instance of CONSsEPLEX with an input graph $G=(V,E)$ and two integers s and k such that $(s+k) \geq 2s+1$ (i.e., $s \leq k-1$). Given an ordering of vertices in V, i.e., $\langle v_1, v_2, \ldots, v_{|V|}\rangle$, we can divide the problem instance into $|V|$ sub-instances. The i-th ($1 \leq i \leq |V|$) one corresponds to finding a size-constrained s-eplex that includes v_i and excludes $\{v_1, v_2, \ldots, v_{i-1}\}$ from G. It is easy to verify that solving $|V|$ sub-instances solves CONSsEPLEX (since all size-constrained s-eplexes in G must be found in one of the formed sub-instance). According to Lemma 6, all s-eplexes with size equal to $s+k$ have the diameter at least 2 when $s \leq k-1$. Thus, for the i-th sub-instance, the graph G can be reduced to a *smaller* subgraph induced by the 2-hop neighbors of v_i, namely $G_i = G[V_i]$, as below.

$$G_i = G[V_i] \text{ and } V_i = N_G^2(v_i) - \{v_1, v_2, \ldots, v_{i-1}\}, \quad (4)$$

where $N_G^2(v_i)$ is the set of 2-hop neighbors of v_i in G. Then, the i-th sub-instance finds a size-constrained s-eplex from G_i, which can be solved by FastEPX-Rec with $S=(\{v_i\},\emptyset)$ and $U=V_i$ on G_i. Following existing studies [9], [64], we adopt the *degeneracy ordering* of vertices in G for dividing the problem instances. Note that (1) the degeneracy ordering can be computed in linear time $O(|E|)$ [5] and (2) with the degeneracy ordering, the size of reduced subgraph G_i is bounded by δd where δ is the degeneracy of G and d denotes the maximum degree of vertices in G.

Algorithm 6: DCFastEPX.

```
Input: A graph G = (V, E) and two integers s and k
Output: A size-constrained s-eplex H
1  if s ≤ k − 1 then
2      Obtain the degeneracy ordering ⟨v1, v2, ..., v|V|⟩;
3      for vi in {v1, v2, ..., v|V|} do
4          Gi ← G[Vi] based on Equation (4);
5          Refine Gi by the reduction rule;
6          H ←FastEPX-Rec(Gi, (vi, ∅), Vi);
7          return H;
8  else
9      H ← FastEPX(G, s, k);
10     return H;
```

By Lemma 5, we note that the subgraph G_i in the i-th sub-instance can be further reduced by the following rule.

- **Reduction rule on G_i.** For a vertex $u \in V_i$ that is non-adjacent to v_i, we can remove u from G_i if u and v have less than $k-2s$ common neighbors; for a vertex $w \in V_i$ that is adjacent to v_i, we can remove w from G_i if w and v have less than $k-s-2$ common neighbors.

The correctness of above reduction rule follows Lemma 5.

With the divide-and-conquer technique, the algorithm called DCFastEPX is summarized in Algorithm 6. Specifically, there are two cases. First, when $s \leq k-1$ (Lines 1-7), it first computes the degeneracy ordering of vertices in V (line 2) and then divides the problem instance into $|V|$ sub-instances (line 3-7). For the i-th one, it constructs a smaller subgraph $G[V_i]$ (line 4), refines the formed subgraph by the reduction rule (line 5), and finally runs FastEPX-Rec on the refined subgraph (line 6). Second, when $s > k-1$ (Lines 8-10), it directly solves the problem by FastEPX (since Lemmas 6 and 5 do not hold and thus the divide-and-conquer technique is not applicable in this case).

Time Complexity: We consider the time complexity of DSA (Algorithm 4) using DCFastEPX (Algorithm 6) to solve CONS EPLEX in Line 4. The time complexity of Algorithm 4 is determined by solving CONsEPLEX using DCFastEPX up to $n-k+1$ times (Lines 3-5 of Algorithm 4). For DCFastEPX, the number of calls to FastEPX-Rec is at most n times and each execution of FastEPX-Rec costs $O(2^n \cdot m^{1.5})$. Consequently, the overall time complexity is $O(2^n n^2 m^{1.5})$.

Remark: Although the worst-case time complexity of Algorithm 4 is $O(2^n n^2 m^{1.5})$, the algorithm demonstrates significantly better practical runtime performance (as verified in our experiments). This is due to the following reasons: (1) In most cases, the CONSsEPLEX can be identified with $s \ll n$. (2) In DCFastEPX, when $s \leq k-1$, the divide-and-conquer strategy partitions the graph into n smaller subgraphs. Each subgraph G_i, refined by (2), contains at most δd vertices, as detailed in the paper. Here, δ and d are typically much smaller than the total number of vertices n in large real-world datasets. (3) In DCFastEPX, the condition $s \leq k-1$ (Lines 1-7) is frequently triggered when processing real-world datasets, as demonstrated by our extensive experimental evaluations.

Space Complexity: The space complexiyt of DSA is bounded by $(|V|+|E|)$, which is dominated by that of the recursive

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

process FastEPX with the depth of $O(|V|)$ and that of refined graph G_i.

Problem Extensions: There are two important variants of the studied problem. One is the personalized search of MINkTRUSS, as many users are particularly interested in the minimum k-truss relevant to themselves. The second is the top-c search of MINkTRUSS, which aims to find the top c minimum k-trusses in a single query, as extracting one minimum k-truss is not enough for some applications. We note that our proposed algorithms can be easily extended to address the two variants.

Personalized Search: This variant requires a personalized search vertex of q and aims to identify a minimum k-truss containing q. Note that such a minimum k-truss may not exist. To solve it, after obtaining the maximum k-truss in Line 1 of Algorithm 4, we check whether q is still in the remaining graph. If not, we can directly terminate the algorithm since q will not be in any k-truss. Otherwise, we continue the algorithm and keep q in the solution set. In Line 4 of Algorithm 6, we choose q to construct G_q. If we cannot find a size-constrained s-eplex in G_q, we terminate the algorithm directly and return $\emptyset$ since we cannot find a size-constrained s-eplex containing q.

Top-c Search: This variant aims to extract r minimum k-trusses with smallest size. To solve it, when there is a size-constrained minimum s-eplex in Line 6 of Algorithm 6, we can extract $c^* = \binom{|V(H)|}{s+k}$ solutions (without repetitive ones). We continue our algorithm to obtain more solutions until $c^* \geq c$ to get top-c solutions. We can efficiently identify all minimum k-trusses by setting the parameter c to a sufficiently large value. This ensures that all minimum k-trusses are successfully discovered.

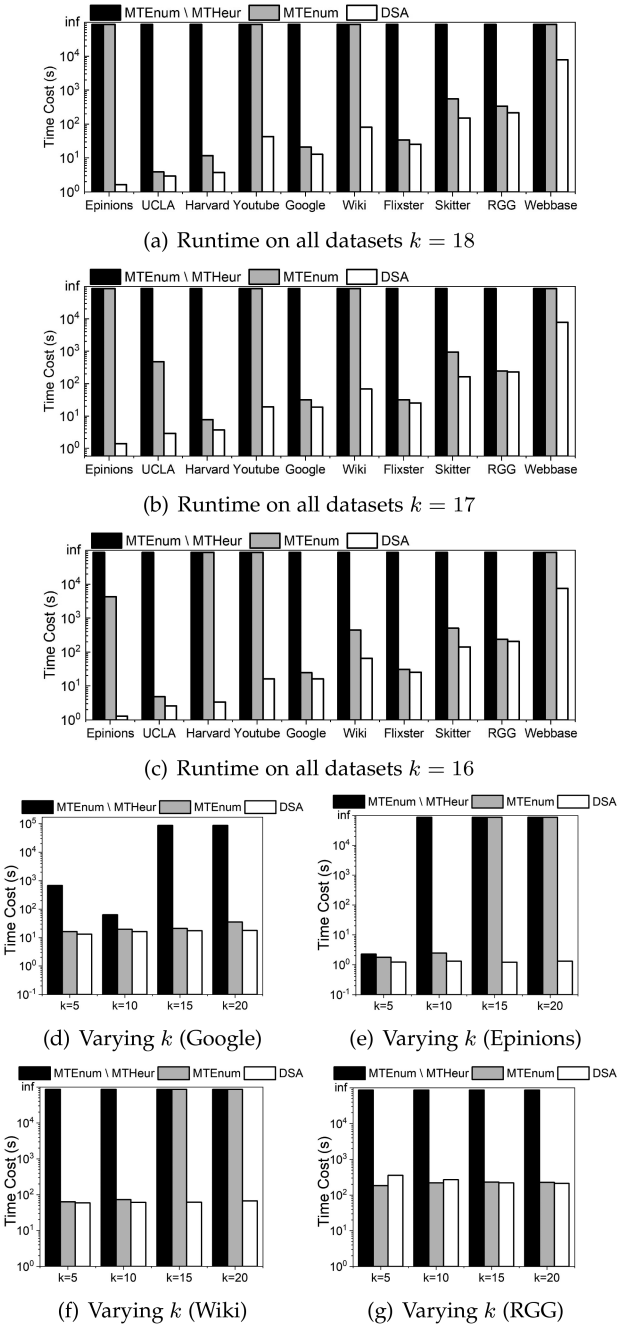
V. Experimental Evaluation

Algorithms: We compare our DSA (Algorithm 4) with the baseline MTEnum (Algorithm 1) and the variants of DSA. All algorithms are implemented in C++ with Intel CPU @ 2.10 GHz and 128 GB RAM. The time limit is set as 24 hours. Our codes are available at https://github.com/Bklight999/MINKTRUSS.

Datasets: We consider 10 publicly available real-world graphs, which are taken from the SNAP (http://snap.stanford.edu) and the NetworkRepository (https://networkrepository.com/). These datasets cover multiple domains, including social and communication networks (Epinions, Youtube, Flixster, Wiki, UCLA, Harvard), web and infrastructure graphs (Google, Webbase, Skitter), as well as a synthetic graph (RGG). The statistics are summarized in Table II, where n, m, d_{avg}, and k_{max} are the number of vertices, the number of edges, the average degree, and the largest k such that a k-truss exists, respectively.

TABLE II
Statistics of All Datasets

Dataset	n	m	d_{avg}	k_{max}	Degeneracy
Epinions	75,879	405,740	10.6	33	170
UCLA	20,453	747,604	73.0	52	240
Harvard	15,126	824,617	109.0	42	352
Youtube	1,134,890	2,987,624	2.6	19	390
Google	875,713	4,322,051	9.9	44	350
Wiki	2,394,385	4,659,565	1.9	53	834
Flixster	2,523,386	7,918,801	6.3	30	758
Skitter	1,696,415	11,095,298	13.1	68	504
RGG	16,777,216	132,557,200	15.0	21	50
Webbase	115,657,290	1,019,903,190	14.8	1,507	21,366



(a) Runtime on all datasets $k = 18$

(b) Runtime on all datasets $k = 17$

(c) Runtime on all datasets $k = 16$

(d) Varying k (Google)

(e) Varying k (Epinions)

(f) Varying k (Wiki)

(g) Varying k (RGG)

Fig. 3. Runtime comparison of different methods.

A. Efficiency Evaluation

Against Baselines: We present the running time of the algorithms DSA (Algorithm 4), MTEnum (Algorithm 1), and MTEnum\MTHeur (Algorithm 1 without using the heuristic algorithm) across all datasets, as illustrated in Fig. 3(a)–(c), with $k = 18, 17, 16$. Overall, the performance of MTEnum is better than that of MTEnum\MTHeur, while DSA demonstrates superior performance over both. Specifically, MTEnum\MTHeur

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

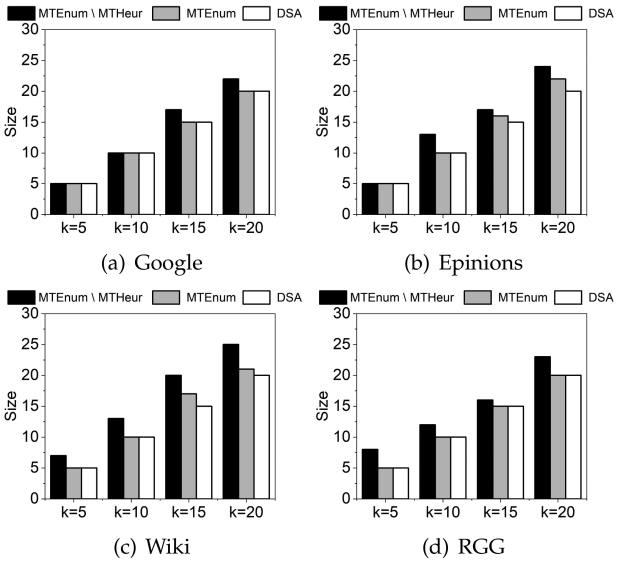


Fig. 4. Solution size comparison of different methods.

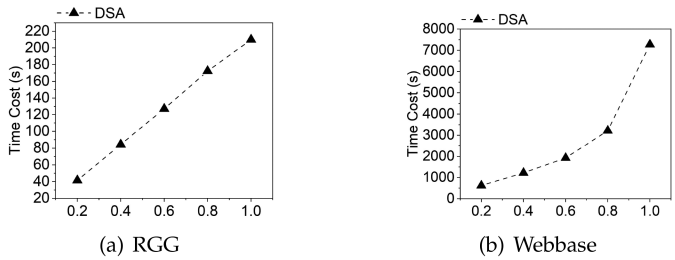


Fig. 5. Scalability test with $k = 15$.

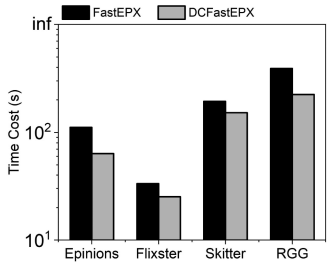


Fig. 6. Efficiency: FastEPX and DCFastEPX.

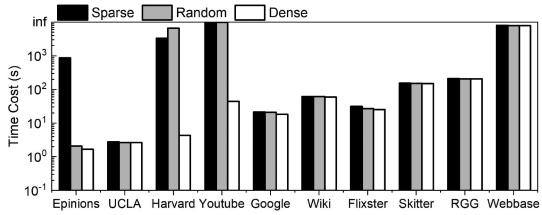


Fig. 7. Evaluation on different query types in the personalized search variant.

exceeds the time limit on all datasets due to its naive branching strategy and large candidate set. In contrast, the heuristic algorithm used in MTEnum provides a powerful upper bound that effectively prunes unpromising branches. Nonetheless, MTEnum may still exceed the time limit when MTHeur (Algorithm 2) fails to give a desirable upper bound. Compared with the baselines, our DSA solves the minimum k-truss problem in a more efficient way. Take the Epinions network as an example, our algorithm is more than five orders of magnitude faster than MTEnum\MTHeur and MTEnum. This is achieved by the carefully designed decomposition-based framework with customized branch-and-bound algorithms.

Furthermore, we compare all methods by varying the value of k across four representative datasets and report the running time in Fig. 3(d)–(g). (1) MTEnum\MTHeur cannot handle the majority of datasets within the time limit, which shows the effectiveness of MTHeur. (2) MTEnum has the running time increase as k grows. This is because MTHeur obtains a poor estimation of the upper bound when k is large. (3) DSA performs the best among all settings. Besides, the speedups achieved by DSA grow as k increases, which indicates its scalability.

In addition, we evaluate the sizes of the k-truss obtained by the three different algorithms in Fig. 4. We set $k = 5, 10, 15, 20$ and perform the experiment on the four representative graphs. Note that if any algorithm exceeds the time limit, we report its found k-truss (not necessarily the minimum one). We have the following observations: (1) DSA consistently finds the minimum k-truss while also being the fastest algorithm. This highlights its effectiveness as well as efficiency. (2) MTEnum demonstrates superiority by generating a smaller k-truss more rapidly than MTEnum \ MTHeur. This indicates the effectiveness of our proposed heuristic method.

Scalability Testing: To verify the scalability of DSA, we consider two large graphs, i.e., RGG and Webbase, by randomly sampling vertices from 20% to 100% of the graph. For each sample, we generate the corresponding subgraph and measure the running time. As in Fig. 5, the runtime of DSA increases consistently with the growth of the graph size. This observation confirms the scalability of DSA when handling graphs of varying sizes.

Comparison between FastEPX and DCFastEPX: We compare the performance of DSA by invoking FastEPX (Algorithm 5) or DCFastEPX (Algorithm 6) in Line 4 of DSA (Algorithm 4). The results are shown in Fig. 6. We observe that DCFastEPX outperforms FastEPX, which indicates the effectiveness of the divide-and-conquer strategy and the reductions. This is also aligned with the theortical results.

Different Query Types in the Personalized Search Variant: We evaluate the performance of the adapted version of DSA for the personalized search variant across various query types, using vertex support as a differentiation metric [65]. In particular, the vertex support of a vertex is the largest support of its incident edge. Then, a personalized search vertex is said to be (1) *dense* if it ranks in the top 10% of vertex support of all the vertices, and (2) *sparse* if it ranks in the bottom 10% of vertex support of all the vertices. The results can be seen in Fig. 7. We have the following observations: (1) the adapted version of DSA can produce results for queries of different types within the specified time in most cases. (2) The performance of the adapted version of DSA heavily

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

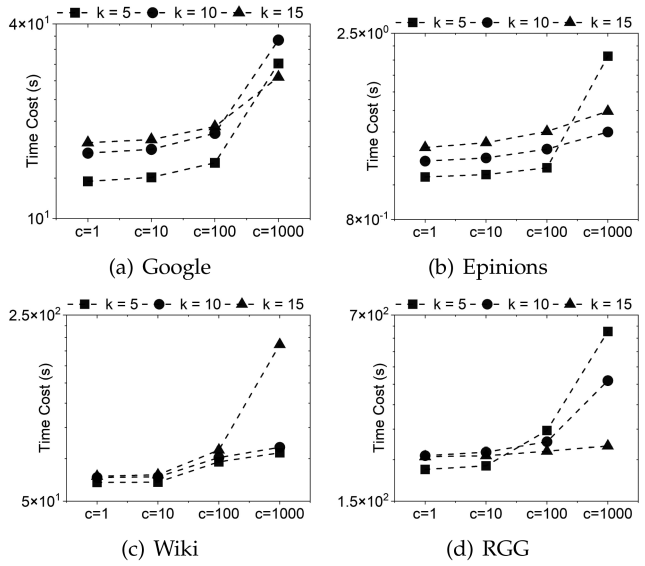


Fig. 8. Runtime by varying c in the top-c search variant.

TABLE III
COMPARISON OF SIZE AND DENSITY AMONG DIFFERENT COHESIVE MODELS (EACH ENTRY REPRESENTS SIZE/DENSITY)

Dataset	Max. k-truss	Min. k-truss	Max. clique
Harvard ($k = 40$)	269 / 0.282	42 / 0.994	39 / 1.0
Youtube ($k = 18$)	387 / 0.164	20 / 0.979	17 / 1.0
Skitter ($k = 68$)	185 / 0.481	74 / 0.994	67 / 1.0
Epinions ($k = 24$)	359 / 0.214	26 / 0.978	23 / 1.0
UCLA ($k = 52$)	68 / 0.965	54 / 0.997	51 / 1.0

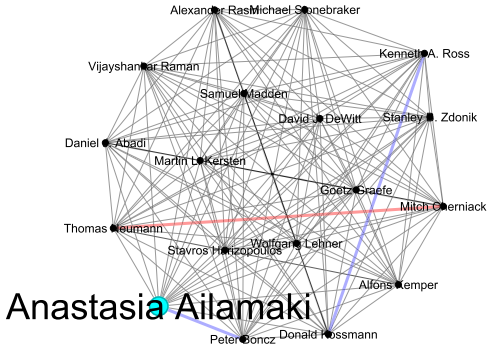


Fig. 9. Citation network prediction using MIN16TRUSS.

depends on the number of iterations over s. Specifically, personalized searches with dense vertices, which typically involve vertices with higher connectivity, often complete faster due to their smaller result sets, requiring fewer iterations. On the other hand, personalized searches with sparse vertices, characterized by vertices with lower connectivity, require more iterations due to their larger result sizes. However, in cases where the scale of results for dense and sparse queries is comparable, the running time is similar, demonstrating a similar number of iterations over s. This evaluation highlights the adaptability of DSA in handling the personalized search variant.

Varying c in the Top-c Search Variant: We next evaluate the top-c search variant by varying the value of c. We record the average running time of the adapted version of DSA algorithm across the four representative graphs. In particular, we set different values of c to 1, 10, 100, and 1000, while fixing k at 5, 10, and 15. According to the results shown in Fig. 8, we can find that the running time of the adapted version of DSA remains stable even as c increases. This stability is due to the fact that the efficiency of our decomposition-based framework, which can identify multiple minimum k-trusses within the same problem instance, as outlined in Section IV.

B. Effectiveness Evaluation

Model Comparison: We first compare the size and density of the maximum k-truss, minimum k-truss, and maximum clique in Table III. From the table, we observe the followings: (1) The minimum k-truss demonstrates greater cohesiveness than the maximum k-truss. (2) The minimum k-truss differs significantly from the clique model. Notably, identifying the minimum k-truss can uncover large cohesive subgraphs that the clique model fails to capture, as large cliques are seldom found in real-world datasets.

Case Study 1. Citation Network Prediction: We consider the case study of author relationship prediction using the minimum k-truss model. To do so, we extract a graph based on the citation data obtained from the AMiner Academic Citation Dataset hosted on Kaggle.[1] The dataset has over 33 million citation relationships between 9.8 million papers published between 1515 and 2019. We focus on the period between 2003 and 2013. Specifically, each vertex of the graph corresponds to an author, and two vertices are connected by an edge if the corresponding authors have collaborated at least 5 times. We then extract the minimum 16-truss subgraph centered around the author Anastasia Ailamaki from the graph, and the minimum 16-truss is shown in Fig. 9, consisting of 18 vertices (authors) and 150 edges (collaborations). Based on the obtained minimum 16-truss, we have the following observations. **First**, the minimum 16-truss corresponds to a well-known community of database researchers that frequently collaborate, as evidenced by the extracted subgraph which contains several highly influential authors in the fields of database and data mining. Their close collaborative connections over the years have formed an important academic cluster that has been driving advancements in query processing, data analytics, and related areas. **Second**, with the constructed minimum 16-truss, we predict that the collaborative network would further strengthen and become a fully connected clique with the addition of only 3 more connections in 2018. Moreover, using the AMiner Academic Citation Dataset containing millions of publication records, our prediction demonstrates an accuracy rate of 66.67%, as 2 out of the 3 additional connections indeed occur in 2018. While one connection is missed in our prediction, the obtained high prediction rate still provides further confirmation of the effectiveness and practical application potential of MIN16TRUSS.

Case Study 2. Scholar Collaboration: In this case study, we focus on the personalized search variant of the proposed minimum

[1]https://www.kaggle.com/datasets/kmader/aminer-academic-citation-dataset

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

(a) 7-truss (b) Minimum 7-truss

Fig. 10. Scholar collaboration using MIN7TRUSS.

k-truss cohesive subgraph model. To achieve, we first construct a graph based on the *DBLP*[2] dataset. Specifically, the graph construction is that (1) each vertex in the graph corresponds to an author in *DBLP*; and (2) two vertices are connected by an edge if they have collaborated with each other at least 7 times in *DBLP*, i.e., they have co-authored at least 7 papers. Then, we focus on a well-established author, Dacheng Tao, and explore potential collaborations with other scholars based on the information of those who previously co-published papers with him. To facilitate this process, we use the traditional k-truss model and the proposed minimum k-truss model respectively. Fig. 10(a) shows the 7-truss extracted from the graph, which has 44 vertices, 200 edges, and a density of 0.211; while Fig. 10(b) presents the minimum 7-truss extracted from the graph, which has 9 vertices, 33 edges, and a density of 0.917. In the case study of scholar collaboration, we can find that the minimum 7-truss model is better than the 7-truss model due to the following reasons: (1) the minimum 7-truss has fewer irrelevant vertices (i.e., authors); (2) the minimum 7-truss has a much higher density. With the help of the minimum 7-truss, we can predict the existence of 8 potential co-authors with Dacheng Tao, as in Fig. 10(b).

Case Study 3. Protein Complex Prediction: We conduct a case study on Protein Complex Prediction using a Protein-Protein Interaction (PPI) network. We compare our proposed Minimum k-Truss model against a Maximum k-Truss baseline (which seeks the largest connected k-truss). Following standard practices in protein complex prediction [45], [46], we evaluate performance using Geometric Accuracy, a widely used metric in protein complex prediction [45].

Let $C = C_1, \ldots, C_p$ be the set of ground truth protein complexes, and $P = P_1, \ldots, P_q$ be the set of predicted complexes (the k-trusses). PPV is defined as follows:

$$PPV = \frac{\sum_{j=1}^{q} \max_{i=1}^{p}(|C_i \cap P_j|)}{\sum_{j=1}^{q} \sum_{i=1}^{p} |C_i \cap P_j|} \quad (5)$$

The comparison results are shown in Table IV. The data strongly supports the validity of the minimum size constraint. The Minimum k-Truss model consistently achieves much higher PPV compared to the maximum k-truss. For example, at $k = 4$, the PPV of Minimum k-Truss reaches 0.752, whereas Maximum k -truss drops to 0.471. This suggests that maximizing size tends to merge distinct functional modules into a single large component, thereby introducing many irrelevant proteins (false positives). Moreover, the Minimum model identifies a larger number of distinct, small-scale structures (e.g., 222vs. 12 at $k = 4$), which better reflects the biological reality that cells typically contain many small protein complexes rather than a few giant ones.

[2]https://dblp.uni-trier.de/xml/

TABLE IV
CASE STUDY OF PROTEIN COMPLEX PREDICTION

Model	SNS	PPV	Accuracy	# k-trusses
Min-5-Truss	0.055	0.952	0.229	67
Min-4-Truss	0.262	0.752	0.444	222
Min-3-Truss	0.406	0.616	0.500	642
Max-5-Truss	0.064	0.786	0.225	7
Max-4-Truss	0.295	0.471	0.373	12
Max-3-Truss	0.603	0.324	0.442	41

TABLE V
MEMORY CONSUMPTION ON DIFFERENT DATASETS (MB). SYMBOL "-" DENOTES TIMEOUTS/FAILURES

Dataset	ENUM	DSA
Epinions	-	19.16
UCLA	126.07	29.14
Harvard	137.18	41.24
Youtube	678.86	152.28
Google	892.43	207.05
Wiki	-	252.12
Flixster	1,770.82	381.45
Skitter	-	496.96
RGG	25,243.30	5,419.30
Webbase	-	16,826.30

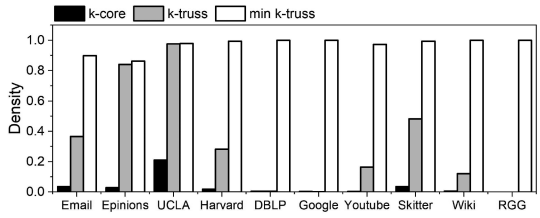


Fig. 11. Density comparison of models with $k = k_{max}$.

C. Additional Experiments

Memory Usage: We report the peak memory usage in Table V. Clearly, DSA has smaller memory usage. In particular, ENUM runs out of memory on several large datasets (e.g., Skitter and Webbase), whereas DSA can handle all datasets.

Comparison with STCS [65]: We conduct additional experiments to compare our method with STCS [65] in terms of both effectiveness and efficiency.

To evaluate the effectiveness, we measure the density of the subgraphs returned by three different k-truss models: the personalized search version of the maximum k-truss, the size-constrained truss community (STCS), and the minimum k-truss. We set $k = k_{max}$ for all graphs. For STCS, the parameters are set to $l = k_{max} - 9$ and $h = k_{max}$. We randomly select 100 query nodes and report the average running time in Fig. 11. We observe that the minimum k-truss consistently achieves higher density than the other models. This demonstrates that the minimum k-truss is particularly effective at identifying dense subgraphs

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

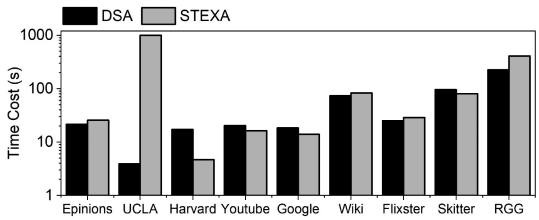


Fig. 12. Efficiency comparison with STEXA ($k = k_{max}$).

TABLE VI
PERFORMANCE OF DSA UNDER DIFFERENT s

Dataset	k	Min k-truss Size	Gap s	DSA (s)	Baseline (s)
UCLA	52	54	2	3.93	> 1000
Harvard	42	48	6	17.28	> 1000
Youtube	19	20	1	20.33	> 1000
Skitter	68	74	6	96.23	> 1000

TABLE VII
RUNNING TIME (S) OF DSA UNDER DIFFERENT GRAPH DENSITIES ($|V| = 10,000$)

Density	0.02	0.04	0.06	0.08	0.1
Running time (s)	6.03	54.29	53.86	85.83	121.91

and cannot be replaced by simply searching for a maximum k-truss with a large k value.

To evaluate the efficiency, we compare STEXA (the algorithm proposed in [65] to solve STCS) with our DSA on the datasets used in the experiment. We follow the same setting and set $k = k_{\max}$ for DSA, while for STEXA we set the size lower bound $LB = k_{\max} - 9$ and upper bound $UB = k_{\max}$. We report the running time in Fig. 12. The results show that both algorithms achieve comparable performance in practice, whereas DSA identifies more cohesive structures (as verified in Fig. 11).

Performance of DSA when $s > 0$: To show the effect of s, we have collected all problem instances where the size of the identified minimum k-truss is strictly larger than k (i.e., $s > 0$), and report the running time in Table VI. We observe that DSA runs slightly slower when s is large, while still achieving the best performance. This is possibly because the size of the minimum k-truss (i.e., $k + s$) increases as s grows, thereby resulting in increased computations.

Running time of DSA by varying graph density: We conduct additional experiments to analyze the impact of graph density on the running time of our proposed method, DSA. Specifically, we generate a systematic and comprehensive series of synthetic Erdős-Rényi (ER) graphs with a fixed size of $|V| = 10,000$ and vary the density from 0.02 to 0.1. The results are reported in Table VII. We observe that the running time of DSA increases with graph density. This is because the size of the maximum k-truss increases as the density grows.

VI. RELATED WORK

Recent years have witnessed significant progress in the field of cohesive subgraph mining [11], [15], [16], [17], [33], [38], [39], [40], [43].

Size-bounded cohesive subgraph search: This problem focuses on finding a cohesive subgraph with a *size constraint*. Li et al. [34] proposed an approximation algorithm with approximation ratio for the minimum k-core problem, while Zhang and Liu [66] later designed an exact solution. Yao et al. [62] considered the problem of finding a connected subgraph with the largest minimum degree, Zhang et al. [65] extended this by maximizing minimum edge support. Zhang et al. [68] also introduced a size-bounded cohesive subgraph model for k-cores on bipartite graphs. He et al. [21] studied the problem of finding a connected subgraph with the smallest conductance that contains the query vertex q and has at least l and at most h vertices. The work most relevant to our research is studied by Liu et al. [36], in which they considered the k-truss problem with a specified size upper bound. However, unlike [36], we get the minimum k-truss which is also the most condensed k-truss in the graph. Recently, Liu et al. [41] proposed a randomized algorithm, achieving a 2-approximation for minimal cohesive subgraphs (e.g., k-core, k-truss, and k-ECC). In contrast, we focus on deterministic algorithms to find the exact minimum k-truss.

k-truss: The k-truss has been studied in the literature [2], [13], [25], [26], [51]. Wang and Cheng [55] proposed efficient in-memory and I/O truss decomposition algorithms, while Chen et al. [12] studied a novel higher-order truss decomposition problem. For the community search with k-truss, Akbas and Zhao [37] introduced the concept of k-truss equivalence to effectively model the intrinsic density and cohesion of k-truss community edges. Jiang et al. [32] developed an I/O algorithm for processing k-truss community queries. Sun et al. [53] studied the truss maximization problem and proposed adaptive methods with more k-truss edges. Zhang et al. [65] proposed a query-based size-constrained k-truss model, which is the closest work to ours. Furthermore, it is worth noting that the k-truss model has also been studied in bipartite graphs [56], dynamic graphs [24], [30], [52], [67], weighted graphs [69], uncertain graphs [27], and directed graphs [35], [37].

VII. CONCLUSION AND FUTURE WORK

In this paper, we studied the Minimum k-Truss (MINk-TRUSS) problem. We proved its NP-hardness and proposed a new framework called DSA, which transforms the MINkTRUSS problem into a series of sub-problems of finding edge-based s-plexes. The latter problem can be solved efficiently via the newly-designed DCFASTEPX algorithm. Finally, extensive experiments were conducted on real-world datasets, and the results demonstrated that our approach achieves a speedup of up to five orders of magnitude compared to the baseline enumeration method. An interesting research direction is to parallelize the proposed method.

ACKNOWLEDGMENT

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Abello, P. M. Pardalos, and M. G. C. Resende, "On maximum clique problems in very large graphs," *External Memory Algorithms*, vol. 50, pp. 119–130, 1998.

[2] E. Akbas and P. Zhao, "Truss-based community search: A truss-equivalence based indexing approach," *Proc. VLDB Endowment*, vol. 10, no. 11, pp. 1298–1309, 2017.

[3] G. D. Bader and C. W. Hogue, "An automated method for finding molecular complexes in large protein interaction networks," *BMC Bioinf.*, vol. 4, no. 1, pp. 1–27, 2003.

[4] A.-L. Barabasi and Z. N. Oltvai, "Network biology: Understanding the cell's functional organization," *Nature Rev. Genet.*, vol. 5, no. 2, pp. 101–113, 2004.

[5] V. Batagelj and M. Zaveršnik, "An $O(m)$ algorithm for cores decomposition of networks," 2003, *arXiv:cs/0310049*.

[6] K. Bhawalkar, J. Kleinberg, K. Lewi, T. Roughgarden, and A. Sharma, "Preventing unraveling in social networks: The anchored k-core problem," *SIAM J. Discrete Math.*, vol. 29, no. 3, pp. 1452–1475, 2015.

[7] C. Bron and J. Kerbosch, "Algorithm 457: Finding all cliques of an undirected graph," *Commun. ACM*, vol. 16, no. 9, pp. 575–577, 1973.

[8] L. Chang, M. Xu, and D. Strash, "Efficient maximum k-plex computation over large sparse graphs," *Proc. VLDB Endowment*, vol. 16, no. 2, pp. 127–139, 2022.

[9] L. Chang and K. Yao, "Maximum k-plex computation: Theory and practice," *Proc. ACM Manage. Data*, vol. 2, no. 1, pp. 1–26, 2024.

[10] Y. Che, Z. Lai, S. Sun, Y. Wang, and Q. Luo, "Accelerating truss decomposition on heterogeneous processors," *Proc. VLDB Endowment*, vol. 13, no. 10, pp. 1751–1764, 2020.

[11] Y. Chen, J. Zhang, Y. Fang, X. Cao, and I. King, "Efficient community search over large directed graphs: An augmented index-based approach," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 3544–3550.

[12] Z. Chen, L. Yuan, L. Han, and Z. Qian, "Higher-order truss decomposition in graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3966–3978, Apr. 2023.

[13] J. Cohen, "Trusses: Cohesive subgraphs for social network analysis," *Nat. Secur. Agency Techn. Rep.*, vol. 42, no. 4, pp. 84–94, 2008.

[14] M. T. Dittrich, G. W. Klau, A. Rosenwald, T. Dandekar, and T. Müller, "Identifying functional modules in protein–protein interaction networks: An integrated exact approach," *Bioinformatics*, vol. 24, no. 13, pp. i223–i231, 2008.

[15] Y. Fang et al., "A survey of community search over big graphs," *VLDB J.*, vol. 29, pp. 353–392, 2020.

[16] Y. Fang, K. Wang, X. Lin, and W. Zhang, *Cohesive Subgraph Search Over Large Heterogeneous Information Networks*. Berlin, Germany: Springer, 2022.

[17] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu, "Effective and efficient community search over large directed graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2093–2107, Nov. 2019.

[18] J. Gao, J. Chen, M. Yin, R. Chen, and Y. Wang, "An exact algorithm for maximum k-plexes in massive graphs," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 1449–1455.

[19] S. Gao, K. Yu, S. Liu, and C. Long, "Maximum k-plex search: An alternated reduction-and-bound method," *Proc. VLDB Endowment*, vol. 18, no. 2, pp. 363–376, 2024.

[20] S. Gao, K. Yu, S. Liu, C. Long, and X. Zhou, "On searching and querying maximum directed (k, ℓ)-plex," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 8, pp. 4743–4757, Aug. 2025.

[21] Y. He, L. Lin, P. Yuan, R. Li, T. Jia, and Z. Wang, "CCSS: Towards conductance-based community search with size constraints," *Expert Syst. Appl.*, vol. 250, 2024, Art. no. 123915.

[22] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. England U.K.: Psychology Press, 2005.

[23] J. Huang, X. Huang, and J. Xu, "Truss-based structural diversity search in large graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 8, pp. 4037–4051, Aug. 2022.

[24] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1311–1322.

[25] X. Huang and L. V. S. Lakshmanan, "Attribute-driven community search," *Proc. VLDB Endowment*, vol. 9, no. 4, pp. 276–287, 2017.

[26] X. Huang, L. V. S. Lakshmanan, J. X. Yu, and H. Cheng, "Approximate closest community search in networks," *Proc. VLDB Endowment*, vol. 9, no. 4, pp. 276–287, 2015.

[27] X. Huang, W. Lu, and L. V. Lakshmanan, "Truss decomposition of probabilistic graphs: Semantics and algorithms," in *Proc. Int. Conf. Manage. Data*, 2016, pp. 77–90.

[28] H. Jiang, F. Xu, Z. Zheng, B. Wang, and W. Zhou, "A refined upper bound and inprocessing for the maximum k-plex problem," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 5613–5621.

[29] H. Jiang, D. Zhu, Z. Xie, S. Yao, and Z.-H. Fu, "A new upper bound based on vertex partitioning for the maximum k-plex problem," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1689–1696.

[30] J. Jiang, Q. Zhang, R.-H. Li, Q. Dai, and G. Wang, "I/O efficient max-truss computation in large static and dynamic graphs," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 3217–3229.

[31] Y. Jiang, Y. Fang, C. Ma, X. Cao, and C. Li, "Effective community search over large star-schema heterogeneous information networks," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2307–2320, 2022.

[32] Y. Jiang, X. Huang, and H. Cheng, "I/O efficient k-truss community search in massive graphs," *VLDB J.*, vol. 30, pp. 713–738, 2021.

[33] V. E. Lee, N. Ruan, R. Jin, and C. Aggarwal, "A survey of algorithms for dense subgraph discovery," in *Managing and Mining Graph Data*, Berlin, Germany: Springer, 2010, pp. 303–336.

[34] C. Li, F. Zhang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Efficient progressive minimum k-core search," *Proc. VLDB Endowment*, vol. 13, no. 3, pp. 362–375, 2020.

[35] X. Liao, Q. Liu, X. Huang, and J. Xu, "Truss-based community search over streaming directed graphs," *Proc. VLDB Endowment*, vol. 17, no. 8, pp. 1816–1829, 2024.

[36] B. Liu, F. Zhang, W. Zhang, X. Lin, and Y. Zhang, "Efficient community search with size constraint," in *Proc. Int. Conf. Data Eng.*, 2021, pp. 97–108.

[37] Q. Liu, M. Zhao, X. Huang, J. Xu, and Y. Gao, "Truss-based community search over large directed graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2183–2197.

[38] Y. Liu, H. Huang, K. Yu, S. Liu, and C. Long, "Efficient maximum s-bundle search via local vertex connectivity," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–27, 2025.

[39] Y. Liu, H. Huang, K. Yu, S. Liu, and C. Long, "Minimum k-vertex connected graph search," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 7, pp. 4159–4165, Jul. 2025.

[40] Y. Liu, H. Huang, K. Yu, S. Liu, C. Long, and Z. Gu, "Efficient size-bounded community search, revisited: Frameworks for practical improvements," *Proc. ACM Manage. Data*, vol. 3, no. 6, pp. 1–26, 2025.

[41] Y. Liu, J. Liu, M. Zhang, and Y. Zhou, "Extracting minimum cohesive subgraphs from large graphs," *SSRN Electron. J.*, no. 4734597, 2024, doi: 10.2139/ssrn.4734597.

[42] R. D. Luce and A. D. Perry, "A method of matrix analysis of group structure," *Psychometrika*, vol. 14, no. 2, pp. 95–116, 1949.

[43] F. D. Malliaros, C. Giatsidis, A. N. Papadopoulos, and M. Vazirgiannis, "The core decomposition of networks: Theory, algorithms and applications," *VLDB J.*, vol. 29, pp. 61–92, 2020.

[44] F. D. Malliaros and M. Vazirgiannis, "To stay or not to stay: Modeling engagement dynamics in social graphs," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2013, pp. 469–478.

[45] T. Nepusz, H. Yu, and A. Paccanaro, "Detecting overlapping protein complexes in protein-protein interaction networks," *Nature Methods*, vol. 9, no. 5, pp. 471–472, 2012.

[46] D. Peleg, I. Sau, and M. Shalom, "On approximating the D-girth of a graph," *Discrete Appl. Math.*, vol. 161, no. 16/17, pp. 2587–2596, 2013.

[47] X. Qi, W. Tang, Y. Wu, G. Guo, E. Fuller, and C.-Q. Zhang, "Optimal local community detection in social networks based on density drop of subgraphs," *Pattern Recognit. Lett.*, vol. 36, pp. 46–53, 2014.

[48] P. Rozenshtein, A. Anagnostopoulos, A. Gionis, and N. Tatti, "Event detection in activity networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 1176–1185.

[49] S. B. Seidman, "Network structure and minimum degree," *Social Netw.*, vol. 5, no. 3, pp. 269–287, 1983.

[50] S. B. Seidman and B. L. Foster, "A graph-theoretic generalization of the clique concept," *J. Math. Sociol.*, vol. 6, no. 1, pp. 139–154, 1978.

[51] Y. Shao, L. Chen, and B. Cui, "Efficient cohesive subgraphs detection in parallel," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 613–624.

[52] Z. Sun, X. Huang, Q. Liu, and J. Xu, "Efficient star-based truss maintenance on dynamic graphs," *Proc. ACM Manage. Data*, vol. 1, no. 2, pp. 1–26, 2023.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.

[53] Z. Sun, X. Huang, C. Piao, C. Long, and J. Xu, "Adaptive truss maximization on large graphs: A minimum cut approach," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 3270–3282.
[54] A. Tian, A. Zhou, Y. Wang, and L. Chen, "Maximal D-truss search in dynamic directed graphs," *Proc. VLDB Endowment*, vol. 16, no. 9, pp. 2199–2211, 2023.
[55] J. Wang and J. Cheng, "Truss decomposition in massive networks," *Proc. VLDB Endowment*, vol. 5, no. 9, pp. 812–823, 2012.
[56] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Vertex priority based butterfly counting for large-scale bipartite networks," *Proc. VLDB Endowment*, vol. 12, pp. 1139–1152, 2019.
[57] Z. Wang, Y. Zhou, C. Luo, and M. Xiao, "A fast maximum k-plex algorithm parameterized by the degeneracy gap," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 5648–5656.
[58] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998.
[59] H. Xia, K. Yu, S. Liu, C. Long, and X. Zhou, "Maximum degree-based quasi-clique search via an iterative framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2025, pp. 3285–3296.
[60] M. Xiao, W. Lin, Y. Dai, and Y. Zeng, "A fast algorithm to compute maximum k-plexes in social network analysis," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 919–925.
[61] T. Xu, Z. Lu, and Y. Zhu, "Efficient triangle-connected truss community search in dynamic graphs," *Proc. VLDB Endowment*, vol. 16, no. 3, pp. 519–531, 2022.
[62] K. Yao and L. Chang, "Efficient size-bounded community search over large networks," *Proc. VLDB Endowment*, vol. 14, no. 8, pp. 1441–1453, 2021.
[63] K. Yu and C. Long, "Fast maximal quasi-clique enumeration: A pruning and branching co-design approach," *Proc. ACM Manage. Data*, vol. 1, no. 3, pp. 211:1–211:26, 2023.
[64] K. Yu and C. Long, "Maximum k-biplex search on bipartite graphs: A symmetric-BK branching approach," *Proc. ACM Manage. Data*, vol. 1, no. 1, pp. 1–26, 2023.
[65] F. Zhang, H. Guo, D. Ouyang, S. Yang, X. Lin, and Z. Tian, "Size-constrained community search on large networks: An effective and efficient solution," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 356–371, Jan. 2024.
[66] Q. Zhang and S. Liu, "Efficient exact minimum k-core search in real-world graphs," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 3391–3401.
[67] Y. Zhang and J. X. Yu, "Unboundedness and efficiency of truss maintenance in evolving graphs," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 1024–1041.
[68] Y. Zhang, K. Wang, W. Zhang, W. Ni, and X. Lin, "Size-bounded community search over large bipartite graphs," in *Proc. Int. Conf. Extending Database Technol.*, 2024, pp. 320–331.
[69] Z. Zheng, F. Ye, R.-H. Li, G. Ling, and T. Jin, "Finding weighted k-truss communities in large networks," *Inf. Sci.*, vol. 417, pp. 344–360, 2017.
[70] Y. Zhou, S. Hu, M. Xiao, and Z.-H. Fu, "Improving maximum k-plex solver via second-order reduction and graph color bounding," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 12453–12460.
[71] Y. Zhou, Y. Fang, W. Luo, and Y. Ye, "Influential community search over large heterogeneous information networks," *Proc. VLDB Endowment*, vol. 16, no. 8, pp. 2047–2060, 2023.

Qifan Zhang received the BEng degree from the Harbin Institute of Technology, Shenzhen, China. His research focuses on database and graph data management.

Yang Liu is currently working toward the PhD degree with the Harbin Institute of Technology, Shenzhen, China. His research interests include database, graph data mining, and graph data management.

Kaiqiang Yu received the BEng degree from Shandong University, China, and the PhD degree from Nanyang Technological University, Singapore. He was a postdoctoral research fellow with the College of Computing and Data Science, Nanyang Technological University. He is currently an assistant professor with the College of Computer, Nanjing University. His research focuses on graph data mining and management.

Shengxin Liu received the PhD degree from the City University of Hong Kong, China. He is currently an associate professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. He was a postdoctoral research fellow with Nanyang Technological University, Singapore. His research interests include graph data management and computational social choice. He was the recipient of Outstanding Student Paper Award at AAAI 2020.

Cheng Long (Senior Member, IEEE) received the BEng degree from the South China University of Technology, China, in 2010, and the PhD degree from the Hong Kong University of Science and Technology, Hong Kong, in 2015. From 2016 to 2018, he was a lecturer with Queen's University Belfast, U.K. He is currently an associate professor with the College of Computing and Data Science, Nanyang Technological University. His research interests include data management and data mining.

Xun Zhou (Senior Member, IEEE) received the PhD degree in computer science from the University of Minnesota in 2014. In 2023, he was a tenured associate professor with the Tippie College of Business, University of Iowa, USA. He is currently a professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. He has authored or coauthored more than 100 papers in his research interests which include spatial and spatiotemporal data mining, machine learning, Big Data analytics, and smart cities. He was the co-editor-in-chief of the *Encyclopedia of GIS*, 2nd Edition. He was the recipient of five best paper awards, including the Best Paper Award from ICDM'21, Best Applied Data Science Paper Award from SDM'19, and US NSF CISE CRII Award in 2016.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 12:40:01 UTC from IEEE Xplore. Restrictions apply.